中国科学技术大学
University of Science and Technology of China

9P-Walkers

基于 mini9P 的 MCU 集群分布式系统
中期汇报

报告人：霍斌
项目组成员：董宇皓、霍斌、刘亦航、刘子源、武文韬

中国科学技术大学
OSH-2026 课程项目

2026 年 4 月 27 日

大纲

1. What：我们到底要做什么
2. Why：为什么这样设计是合理的
3. How：准备怎样把它做成

大纲

1. What：我们到底要做什么
2. Why：为什么这样设计是合理的
3. How：准备怎样把它做成

多 MCU 的统一资源视图

- 单个 MCU 上的调度、驱动和文件系统已经较成熟，但多 MCU 协同时往往接口分裂、控制入口分散。
- 真正困难的不是某个设备怎么读写，而是多个节点上的设备、存储和计算任务如何被统一命名、统一访问、统一调度。
- 因此，本项目关注的是一个轻量级分布式系统原型，而不是再做一个单板 OS。

多 MCU 的统一资源视图

- 单个 MCU 上的调度、驱动和文件系统已经较成熟，但多 MCU 协同时往往接口分裂、控制入口分散。
- 真正困难的不是某个设备怎么读写，而是多个节点上的设备、存储和计算任务如何被统一命名、统一访问、统一调度。
- 因此，本项目关注的是一个轻量级分布式系统原型，而不是再做一个单板 OS。

项目边界

不是复刻完整 Plan 9、Inferno 或 ROS 2，而是在 MCU 资源约束和课程周期内，验证“统一命名空间 + 轻量远程文件协议 + 主从协同”这条路线能否跑通。

What-2：最终系统要交付什么闭环

用户最终看到的系统

- `cat /mcu1/dev/temperature`
- `echo 100 > /mcu2/dev/motor/speed`
- `cat /mcu3/compute/jacobi/result`

要交付的核心能力

- 统一路径访问
- 跨节点设备读写
- 固定计算任务触发与取回结果

系统形态

- Master 负责命名空间、Shell、Lua、后续 Web
- Slave 负责设备访问、本地存储、计算执行
- 节点间通过 mini9P 做远程文件访问

明确的非目标

- 不实现完整 9P2000
- 不引入完整 ROS 2 图与消息系统
- 不在当前阶段追求完全去中心式架构

大纲

1 What：我们到底要做什么

2 Why：为什么这样设计是合理的

3 How：准备怎样把它做成

Why-1：为什么这个问题值得做

课程价值

- 资源抽象与统一命名空间
- 本地文件系统与远程访问协议协同
- 多节点通信与任务调度裁剪实现
- 受限系统中的分层与边界控制

工程价值

- 机器人和嵌入式系统中，本来就存在“上层控制器 + 多个 MCU 执行节点”的常见形态。
- 如果接口仍是专用串口命令，系统会随节点数增长迅速失控。
- 因此，需要一个比“设备私有 RPC”更统一、更可扩展的资源视图。

调研结论：Plan 9 / Inferno 给出的启发

- 设备与服务都可以被稳定地抽象为文件。
- 远程访问可以围绕少数通用操作展开，而不是为每类设备单独设计协议。
- 用户应看到聚合后的命名空间，而不是散乱的控制接口。

工程后果

- Shell 直接可用，演示和调试都直观。
- 外设文件、状态文件、计算任务文件可以共用一套访问语义。
- 接口数量随节点增长更可控，不会演化成一堆专用命令集合。

结论

文件抽象不是“为了好看”，而是为了让设备控制、状态查询和任务执行都落到统一接口上。

现有成熟系统过于复杂，我们考虑实现简化模型

- **ROS / micro-ROS**：适合作为上层分层和现实部署形态的参考，但它们擅长的是消息图与编排，不是“统一文件命名空间”。
- **完整 9P2000**：语义值得借鉴，但对 MCU 的 RAM、Flash、无 MMU 环境来说过重；我们只保留当前闭环需要的操作集合。
- **完全去中心式**：理论上更分布式，但会把发现、路由、一致性和故障恢复的复杂度过早引入系统。

现有成熟系统过于复杂，我们考虑实现简化模型

- **ROS / micro-ROS**：适合作为上层分层和现实部署形态的参考，但它们擅长的是消息图与编排，不是“统一文件命名空间”。
- **完整 9P2000**：语义值得借鉴，但对 MCU 的 RAM、Flash、无 MMU 环境来说过重；我们只保留当前闭环需要的操作集合。
- **完全去中心式**：理论上更分布式，但会把发现、路由、一致性和故障恢复的复杂度过早引入系统。

因此当前选择是

逻辑上采用主控集中、从机确定执行；协议上采用mini9P；物理链路上先从UART 单链路闭环出发，后续再考虑总线或环形演进。

Why-4：为什么判断这个方案在课程周期内可行

依据 1：代码底座已存在

- mini9P 协议规范已成文
- 主控 client 已实现编解码和 API
- 主控已能启动 Lua 和 shell

依据 2：从机基础已具备

- USART2 / SDIO 初始化已完成
- littlefs 自检已能建目录、写读、校验
- 能周期输出启动与状态报告

依据 3：验证路径清晰

- 先协议层自测
- 再单节点主从闭环
- 再统一命名空间
- 最后扩展展示层

大纲

1. What：我们到底要做什么
2. Why：为什么这样设计是合理的
3. How：准备怎样把它做成

How-1：系统分层与职责划分

Master：控制平面

- Shell / Lua / 后续 Web 作为统一入口
- cluster_vfs 负责 `/mcuN/...` 命名空间聚合
- mini9P client 负责远程访问协议收发
- 调度逻辑和复杂状态主要留在主控侧

$\Longleftrightarrow$

UART

Slave：执行平面

- mini9P server 解析请求
- 虚拟文件树路由到设备或计算节点
- littlefs 提供本地持久化基底
- 从机侧坚持静态、确定、可预测

How-2：路径、协议与任务是如何落地的

路径模型

- 主控对外暴露统一路径：`/mcu1/dev/temperature`
- mini9P 只处理节点本地路径：`/dev/temperature`
- 也就是说，`/mcuN` 这一层聚合由 Master 完成

协议模型

- 保留 attach / walk / open / read / write / stat / clunk
- 帧头 + 长度 + CRC 便于串口同步与恢复
- 不把消息集合扩得超过当前闭环需求

任务模型

- 长任务不走阻塞式 RPC
- 向命令文件写参数，立即返回
- 通过状态文件轮询执行进度
- 通过结果文件读取最终输出

原因

这样可以避免在没有复杂会话控制和 `flush` 的前提下，把链路做成难以恢复的长事务。

How-3：分阶段推进，而不是一次把系统做满

阶段	目标	当前状态	验收标准
协议基线	协议规范、编解码、client API	已完成大半	请求与响应语义稳定
单节点闭环	UART + server + 最小文件树	当前重点	主控可 attach/stat/read/write 从机
统一命名空间	`/mcuN/...` 聚合访问	待接入	Shell 能按统一路径访问资源
展示层扩展	计算 demo、Lua、Web	后续增强	完成课程展示闭环

分阶段的意义

每一阶段都要有独立可验证的结果，这样即使后续增强项未全部完成，核心闭环也能成立。

已完成的底座

- 主控侧已完成 mini9P 帧编解码与 client 接口
- 主控入口已能启动 Lua、boot demo shell 与 REPL
- Shell 已支持 `help`、`heap`、`lua`、`m9p`、`reboot`
- 从机已完成启动、自检、littlefs 基础验证与状态上报

尚待补齐的关键点

- 真实 UART transport 还未接到主控 client
- 从机 mini9P server 仍是骨架，尚未完成请求分发
- cluster_vfs 与统一路径映射尚未真正落地
- 设备文件树和计算文件接口还未进入端到端联调

风险

- 方案过度膨胀，偏离最小闭环
- 协议可靠性与联调问题拖慢节奏
- 多节点场景过早引入，导致定位困难
- 从机资源受限，复杂逻辑压垮系统

应对

- 坚持先单节点、再统一路径、最后扩展示层
- 先补 transport、server、最小文件树，不扩消息集合
- 多节点在单节点稳定后再引入
- 把复杂状态留在 Master，从机侧坚持静态缓冲区与固定表项